

Table of Contents

BUILD: - Java	2
What is "Build" in Software Development?	2
DevOps Process.....	2
Java based apps:	2
Applications/Software's are developed using languages like:.....	3
Build Tools:.....	3
Apache Maven	3
Maven is an open-source java based build tool.	3
What is expected from developers?	4
Unit Testing:.....	4
Maven Build Tool:	5
Any software can be available in three options:	5
Build in SDLC Implies:.....	5
Maven Installation:	6
Installation Script:	6
MHD = /opt/maven.....	7
Generally We have:.....	7
Maven: creates deployable packages	8
Maven has 3 lifecycles:	10
mvn package:	11
The main "goals" in Maven include:	11
What are we building from?	12
Tags can be created based on the user preferences	12
Maven Build	12
Interview Question: 1.	13
Ticket:	14
Create artifacts using maven for maven-web-applications.....	14
How can a Specific module be built in maven-enterprise-applications?.....	15
How can we trouble-shoot a fail build in MAVEN?.....	15
What is the function of Tomcat	15
What do we consider in selecting dependencies-->? :	15
Java Repositories:.....	16

BUILD:- Java

What is "Build" in Software Development?

In software development, a build is the process of converting source code (raw code) files into standalone software artifact(s) that can be run on a computer, or the result of doing so.

A 'build' is an attempt to compile all of the source code(raw code | the raw material) for the project into a set of executable binaries, that in turn get packaged into a final product.

If you 'break the build', then the code does not compile or pass appropriate regression tests and the product cannot ship.

DevOps Process

Developers(**code**) -> Test (**Selenium**) -> "Build" -> Qualify QA(**SonarQube**) -> Backup (**Nexus, Amazon S3**) -> Deploy -> Monitor applications.

A "Build" is the process of converting source code into a functional, standalone program that is executable and that can run on a computer or mobile device.

Build tools are therefore used to create deployable packages. i.e. packages that app servers can understand.

Java based apps:

"scr code" + "build" = packages/ artifacts

mvn package = artefacts

.jar (standalone apps)

.war (web apps)

.ear (enterprise apps)

Applications/Software's are developed using languages like:

- Java --> build --> package (.jar, .war, .ear)
- Python
- NodeJS
- .NET
- JavaScript
- Etc.
- html --> no build
- - php --> no build

In our environment, we support Java based projects.

Build Tools:

- Java: "Maven", Gradle, ANt
- .NET: MsBuild, NaNt
- JavaScript: Gulp, Grunt
- Python: PyBuilder
- Ruby: Rake
- NodeJS: nmp

For Java we can use Gradle, ANt and Maven.

Apache Maven

Apache Maven is a software Project Management and comprehension tool. Based on the concept of a "Project Object Model" (POM, pom.xml), Maven can manage a project's Build, Reporting and Documentation from a Central piece of information.

Maven is a build automation tool used primarily for Java projects. Maven can also be used to build and manage projects written in C#, Ruby, Scala, and other languages. The Maven project is hosted by The Apache Software Foundation, where it was formerly part of the Jakarta Project.

Official Website: https://maven.apache.org/
--

Maven is an open-source java based build tool.

1. **Open Source** are Software's that are "free" including their "source codes".
2. **Free Software's** are made available to users free of charge excluding their source code.
3. **Licensed Software's** are paid to use by users excluding their source code.

What is expected from developers?

Maven Project main composition:

1. Source Code = app.java
2. Unit-Test-Cases = appTests.java
3. Build Script POM = pom.xml (Project file)

Unit Testing:

Is the process of writing and running "unit-test-cases" by developers.

An ideal unit-test-Cases file should test all the lines in the code.

```
print ("I need a private jet")
```

```
echo "I need a private jet"
```

unit-test-case

Example:

```
app.sh = echo "Welcome to TechLink Academy"
```

Unit Test Case:

```
#!/usr/bin/env bats

@test "Check if output is 'Welcome to TechLink
Academy'" {
    result="$(echo "Welcome to TechLink Academy")"
    [ "$result" = "Welcome to TechLink Academy" ]
}
```

In this test case:

We use BATS framework to structure our test case.

We run the command echo

```
"Welcome to TechLink Academy"
```

and capture its output in the variable result.

We then check if the output stored in result is equal to the expected output

"Welcome to TechLink Academy"

src = source code = 20k lines

unit-test-Cases = 20k lines

Who writes unit test cases = developers

Who performs/runs unit testing = developers

Maven Build Tool:

<https://maven.apache.org/download.cgi>

MAVEN is an open-source Java BASED Build tool. It was originally developed in java for java codes.

The vendor of Maven is **APACHE**

Any software can be available in three options:

License edition: The software has to be paid for.

Free edition: Software is given for free BUT without source-code. How the software was developed is not shared.

Open-source edition: Implies both the software and the source code are freely available. You can download the source code and develop/add/modify on the existing features

Build in SDLC Implies:

creating deployable packages from raw codes.

compiling

Hello.java --> java codes

hello.py --> python codes

hello.js --> nodeJS codes

hello.sh --> bash shell script

Maven Installation:

<https://maven.apache.org/download.cgi>

Maven can be installed in Any platform:

Linux: RedHat / Ubuntu / CentOS / amazon linux /etc

Windows:

server2012/2016/2019

Windows XP/7/8/10/11

MacOS : Solaris.

Install Maven on a RedHat Linux 9 Server:

Java is a pre-requisite for maven to run

JRE --> JAVA RUNTIME ENVIRONMENT

JDK --> JAVA DEVELOPMENT KIT 1.8+,

JVM will also be installed

<https://maven.apache.org/download.cgi>

Installation Script:

Prerequisite:

EC2 T2.medium instance (4 GB of RAM)

EC2 T2.micro instance (1 GB of RAM)

1. Install Java openJDK 11+ as a pre-requisit for maven to run.

```
$sudo hostnamectl set-hostname maven
```

```
$sudo su - ec2-user
```

```
$cd /opt
```

```
$sudo yum install wget tree unzip git-all -y
```

```
$sudo yum install java-11-openjdk-devel java-1.8.0-openjdk-devel -y
```

```
$java -version
```

```
$git --version
```

2. Download, Extract and Install Maven

```
$sudo wget https://dlcdn.apache.org/maven/maven-3/3.9.8/binaries/apache-maven-3.9.8-bin.zip
$sudo unzip apache-maven-3.9.8-bin.zip
$sudo rm -rf apache-maven-3.9.8-bin.zip
$sudo mv apache-maven-3.9.8/ maven
```

3. Set Environmental Variable - For Specific User eg ec2-user

```
$ vi ~/.bash_profile
export M2_HOME=/opt/maven
export PATH=$PATH:$M2_HOME/bin
$source ~/.bash_profile
```

MHD = /opt/maven

The MHD is where the maven software is extracted.

```
ls /opt/maven : bin -- binary files (mvn)
                conf -- configuration files (settings.xml)
                lib -- library files (jar files)
                LICENSE, README.txt
```

We use maven to build and manage applications for java codes in our environment.

Generally We have:

- ❖ standalone applications - jar [app.jar, hello.jar]
- ❖ web applications - war [app.war]
- ❖ enterprise applications - ear [app.ear]

hello.java:

After using maven to build the following options are possible

hello.java --> hello.jar [stanalone apps]

hello.java --> hello.war [web applications]

hello.java --> hello.ear [enterprise apps]

Maven: creates deployable packages

(jar, war and/OR ear)

1. jar: Standalone Applications

ebay.jar

paypal.jar

tesla.jar

*.jar

2. war: web Applications

boa.war

tesla.war

*.war

3. ear: Enterprise Applications

aa.ear

tesla.ear

td.ear

Standalone Applications – *.jar Java Archives

contains only java code

contains only java classes

jar = .classfiles

maven-standalone-application.jar

app.jar

Web Applications – .war web Archives :

Java code + web content

web content (CSS, HTML, JS, videos, images...,)

Backend code + frontend code

maven-web-app.war

boa.war

Enterprise applications – .ear –:

Enterprise Archive

Multiple Modules of Java code + web content

ear = war(s) + jar(s)

Banking applications

maven-web-application.war

maven-enterprise-application.ear

modules:

login /

accountCreation /

moneyTransfer

Based on our project we are going to create multiple packages either; jar, war or ear packages.

Maven has 3 lifecycles:

1. Clean
2. Site/swagger
3. Default

Lifecycles	Goal(s):
1. Clean mvn clean	clean (mvn clean) delete old builds
2. Site/Swagger	Site: create java classes (byte code) JVM app.java = 'echo DevOps is great' app.class = 012450012354
3. Default	mvn validate: It will validate the project structure and resource files mvn compile: It will compile all java classes and test classes app.java = echo 'JESUS IS LORD' app.class = 0123412547622 - JVM mvn test: It will run the unit test cases (JUNit) mvn package: It will create packages in target directory (* .jar/* .war/* .ear) app.war mvn install: It will store the build artifacts in MAVEN LOCAL REPO default location: .m2/repository mvn deploy: NEXUS It will upload the build artifact into maven-remote-repo

mvn package:

1. validate
2. Compile
3. Run Unit Testing
4. Create the packages [jar/war/ear]

The main "goals" in Maven include:

- COMPILE** - Compiles the source code of the project.
- PACKAGE** - Packages the compiled code into a distributable format, such as a .JAR, .WAR, or .EAR file.
- INSTALL** - Installs the package into the local repository, making it available for other projects.
- DEPLOY** - Copies the final package to the remote repository for sharing with other developers and projects.
- CLEAN** - Deletes the "target directory", removing all build artifacts.

Maven uses a series of phases to execute these goals. Some of the main phases include:

- mvn validate** - Validates the project structure.
- mvn compile** - Compiles the source code.
- mvn test** - Runs tests on the compiled source-code.
- mvn package** - Packages the compiled code into a distributable format. + app
- mvn verify** - Runs checks on the package to verify its integrity.
- mvn install** - Installs the package in the local repository. (target dir | .jar | .war | .ear)
- deploy** - Deploys the final package to a remote repository.

These goals and phases help streamline the build process and ensure consistency across different projects.

What are we building from?

We are creating deployable packages from java codes:

1. src-- source code:
2. Unit-Test-Cases:

JAVA --> JUnit test cases

.Net --> NUnit test cases

C++ --> CppUnit test cases

3. Build Scripts -----> pom.xml:

pom = project object model

xml = extensible mark-up language

It is a dynamic language.

Tags can be created based on the user preferences

index.xml = <hello> Welcome to TechLink Academy </hello>

<rich> You are blessed </rich>

html = hyperText mark-up language

It comes with pre-define tags that can't be changed

changed tags: h1, h2, h3, p,

expected output = Welcome to TechLink Academy

Maven Build

Clone java project from Github -----> [git clone https://github.com/Blessed25/demo-mwa.git](https://github.com/Blessed25/demo-mwa.git)

mvn package

- Validate
- Compile
- Run Unit Testing
- Create Package (.Jar, .War, .Ear)

maven uses plugins and dependencies

Maven has Quality Gates,
maven ensures that test passes before packages are created.:

"Test Results:"

Tests run: 700, Passes:698 Failures: 2, Errors: 0,
Skipped: 0

Interview Question: 1.

Assuming that 698 Testcases pass and 2 fail, what can be done for maven to still do a build?

We can rerun the build and skip unit-testing

```
mvn package -DskipTests  
mvn package -Dmaven.test.skip=true
```

mvn package:

create packages in the target directory

mvn install:

create packages in the both

1. target directory and the
2. mavenLocalRepo

Maven uses plugins and dependencies to validate, compile, run Unit Tests and creates artifacts

These plugins and dependencies are located in either:

1. MavenLocalRepo or
2. maven-remote-repo or
3. maven-proxy-repo or
4. Maven-central repo

Pulling plugins and dependencies from mavenLocalRepo give the best, fastest & most secured Result.

Ticket:

Create artifacts using maven for maven-web-applications

POM (Project Object Modules)

vi pom.xml

Project:

<modelVersion>4.0.0

<groupId>com.techlink

<artifactID>name_of_app

<packaging>war | jar | ear

<version>buildVersion

<name>companyName

<url>company website

properties:

dependencies:

Interview Question:2.

Explain your DevOps Application Management experience?

I'm managing federated web & Enterprise micro-service applications for fintech clients Enterprise applications with over 21 modules

1. MavenEnterpriseApp-ear
2. MavenEnterpriseApp-web
3. MavenEnterpriseApp-ear

How can a Specific module be built in maven-enterprise-applications?

`mvn package -pl MavenEnterpriseApp-web`

Artifacts Management:

Artifacts/packages can be located, uploaded or installed in:

- target directory -- maven server
- Maven-local-repo -- maven server
- maven-remote-repo -- nexus server

Plugins/dependency management:

location

- Maven-local-repo -- maven server
- maven-remote-repo -- nexus server
- maven-proxy-repo -- nexus server
- maven-central-repo -- apache maven/internet

How can we trouble-shoot a fail build in MAVEN?

- Check the logs to understand the Errors
- `mvn -X package` (BUILD in debugging mode)

What is the function of Tomcat

--> an application server

What do we consider in selecting dependencies-->? :

Type of project -- we manage java projects

1. Maven is a java based open source build tool
2. Maven can be install on any platform [Linux/Windows/macOS,Solaris]
3. The Maven server must have atleast 2GB of free space for maven to run.

4. Maven packages can be created by running the commands below:

`mvn package`

`mvn install`

`mvn deploy`

5. Maven uses plugins and dependencies to validate, compile, test and to Build

6. Java 1.8+ must be running on the maven server before maven can work.

7. maven will create jar, war and ear packages for standalone, web and enterprise applications respectively

8. Maven will search for dependencies and plugins from :

1. mavenLocalRepo

2. mavenRemoteRepo

3. mavenProxyRepo

4. maven central repository

9. mavenLocalRepo located in the maven server is the fastest and most secured location for dependencies/plugins.

Java Repositories:

Standalone:

`https://github.com/eayafor/mwa.git`

`git clone --branch master https://github.com/eayafor/mwa.git`

WebApp:

`https://github.com/ayafor-code/javaWebApp.git`

`git clone https://github.com/ayafor-code/javaWebApp.git`

ayafor-code

`ghp_GxnQPp2a8EpZNPk5MxMK6CskNEaFFY4Pd3`

`mvn package`

`/`

`mvn install`

Google Authenticator